

Replica Platform Build Specification

A full technical specification for building a functional replica of the VirtualAccelerate accelerator-management platform on the FinnPayments / FinnVerify VPS.

Prepared for	Ashraf Esmael, Mont Choisy Group / Finnpack
Target deployment	Same VPS as FinnPayments / FinnVerify (Ubuntu 24.04, nginx, FastAPI, React/Vite, SQLite, systemd)
Document version	1.0 — 25 August 2026
Status	Draft for development

1. Purpose & how to read this document

This is a build specification for a **functional replica** of the VirtualAccelerate platform (the SaaS currently used at `app.virtualaccelerate.com`, where Finnpack operates a branch called "Finnpack LTD"). It was produced by studying the running platform directly: navigating every module, inspecting the network/API layer, and reading the live JSON data model through the platform's own API.

It is written so that a developer can build the replica from scratch. It covers:

- What the platform *is* and the end-to-end business process it supports (Sections 2–5)
- A module-by-module functional and UI specification (Section 6)
- The data model and entity relationships (Section 7)
- The API surface and conventions observed on the original (Section 8)
- Authentication, roles and permissions (Section 9), i18n (Section 10), non-functional needs (Section 11)
- The **target technology stack and VPS deployment**, chosen to match FinnPayments (Sections 12–13)
- A phased delivery roadmap (Section 14) and appendices with the raw observed API catalogue, enumerations and field types (Section 15)

Note on fidelity. Everything in Sections 2–9 is derived from the *observed* original system, so it is an accurate description of how the real product behaves and is structured. Sections 12–14 are the *recommended* implementation for your environment, not a copy of the original's internals (the original appears to be a Django/DRF backend; we are deliberately re-implementing on the FastAPI/React stack you already run, so it slots in beside FinnPayments).

2. Executive summary — what VirtualAccelerate is

VirtualAccelerate is a **multi-tenant accelerator / incubator management platform**. It runs the complete lifecycle of an entrepreneurship support programme (an "accelerator" or grant/investment programme), from the moment an entrepreneur applies through to selection, training, mentoring, graduation, and finally investment and portfolio management.

In your case it is used to run **OCIF** ("Track I Support – 6/8 Months" and "Track II Support – 12 Months") cohorts of Jordanian and regional startups: 80 applicants, 79 selected businesses, several cohorts, 12 training courses, mentor evaluations, and an investment committee/portfolio pipeline.

It is best understood as the fusion of five products in one:

1. **An application & selection ATS** — configurable public application forms, an applicant pipeline with custom stages, weighted scoring forms filled in by evaluators, and selection decisions.
2. **A learning management system (LMS)** — multi-language courses with modules, lessons and video content, plus per-entrepreneur progress tracking.
3. **A mentoring & evaluation system** — mentor assignment, mentor review question sets and conclusion question sets, evaluator scoring, and graduation decisions.
4. **An investment / deal-flow CRM** — investment cases moving through deal-flow stages, multi-level approval committees, and portfolio management.
5. **A reporting & collaboration suite** — a rich analytics dashboard, an investment reporting pack (portfolio snapshot, payments schedule, aging analysis, forex, and ~11 detailed financial sections), announcements, a shared calendar, a people directory, and in-app chat.

The whole thing is **multi-tenant**: an *Organization* owns one or more *Branches*, and virtually every record is scoped to a branch (`organization_branch` / `branch_id`). Users switch the active branch from the top-left switcher.

3. Personas & roles

The platform ships five roles (some are "constant"/system roles that always exist per branch; others are custom and can be created):

Role (display)	Code name	Constant?	Who they are / what they do
Administrator	<code>branch_admin</code>	Yes	Branch-level super user. Full access to a branch: programmes, forms, selection, courses, investment, users, reports, settings. This is the persona of the admin dashboard studied here.
Organization Administrator	<code>organization_admin</code>	Yes	Manages the whole organization and its branches (a level above branch admin).
Investment Manager	<code>investment_manager</code>	No (custom)	Manages deal-flow, approvals and portfolio; a custom role showing the RBAC is extensible.
Mentor	<code>mentor</code>	Yes	Assigned to businesses/projects; answers mentor review & conclusion questions; can evaluate/score.
Entrepreneur	<code>entrepreneur</code>	Yes	The applicant/founder. Applies via forms, is enrolled as a business/project, takes courses, is scored, mentored and (potentially) invested in.

Two distinct front-end experiences exist behind one login:

- **Staff / admin experience** — the module-rich dashboard documented here (admins, org admins, investment managers, mentors see role-appropriate subsets).
- **Entrepreneur experience** — a lighter portal where founders complete application/onboarding forms, take their assigned courses, track their own progress, receive announcements and chat with staff/mentors.

Additionally there is a **public / unauthenticated surface**: the application form that entrepreneurs fill in to apply (the "Onboarding Form"), which feeds the selection pipeline.

4. Information architecture (navigation & routes)

Global chrome is a **left sidebar + top bar**. The top bar has: organization/branch switcher (top-left), language selector (i18n), a notifications bell (with unread count), and the user avatar/profile menu. The sidebar groups modules as below. Observed client routes (React SPA, client-side routing) are given in `code font`.

Sidebar (grouped):

- **Home** — `/dashboard/analytics` (dashboards & analytics)
- **Announcements** — `/announcements`
- **Selection Board** — `/selections/investment-board` (the applicant pipeline)
- **Forms** — `/forms` (application forms + scoring forms)
- **Programs** — `/programs`, program detail `/programs-courses/{programId}/`
- **Courses** — `/courses` (LMS)
- **Library** — `/library`
- **Investment Forms** — `/investment-forms`
- **Dealflow** — `/deal-flow`
- **Approval** — `/approval`
- **Portfolio management** — `/portfolio-management`

- **Reports** — `/reports`
- **Calendar** — `/apps/calendar`
- **Directories** — `/directories`
- **Chat** — `/apps/chat`
- **Users** — `/administration/roles-permissions/`
- **Organizations** — `/administration/organizations`
- **Help Center** — `/help-center/`

The SPA uses client-side routing; deep links that aren't the "canonical" route can 404 on hard refresh unless the server serves `index.html` for all app routes. The replica must configure the web server to fall back to `index.html` for unknown non-API paths (standard SPA history-fallback).

5. The end-to-end business process (the core funnel)

This is the spine of the whole system; every module plugs into it.



Step by step:

1. **Apply.** An entrepreneur submits a published **Application Form** tied to a Program. This creates an **Applicant** record (business + founder details) at the first selection stage.
2. **Select.** In the **Selection Board**, staff/evaluators move applicants through configurable **Stages** (e.g. *Track I Onboarding* → *Track I Support* → *Pitch Stage* → *Track II Onboarding* → *Track II Support*). At relevant stages, evaluators complete a **Scoring Form** (weighted questions) producing an *Average Evaluator Score*. Applicant

status is one of *Invited, In Selection, Rejected, Archived*. A **registered** flag tracks whether the founder has created their login.

3. **Enrol.** Selected applicants become **Businesses/Projects** inside a **Program** (a cohort). The program bundles **Courses**, tracks **course progress** and **course score**, holds **Mentor review questions** and **Mentor conclusion questions**, and records a **Graduation** decision per business.
4. **Deliver.** Entrepreneurs take assigned **Courses** (modules → lessons → video/content, with progress). **Mentors** are assigned and complete review/conclusion questions. Businesses accumulate scores.
5. **Invest.** The **Invest** action promotes a business into the **Investment** track: a **Deal-flow investment case** progresses through *Application → Evaluation → Pitching → Selection → Approved*, is graded by **tier** (1–3) and **round** (Pre-seed ... IPO), routed through multi-level **Approval committees** (statuses *In Approval, Revision, Approved, Rejected*), and finally managed in **Portfolio management** with full financial **Reports**.

Everything above is filtered/aggregated in the **Dashboards** and surfaced through **Announcements, Calendar, Directory and Chat** for collaboration.

6. Module-by-module functional specification

For each module: purpose, key UI, features/actions, and the backing API (see Section 8/15 for the full endpoint catalogue). All list endpoints are branch-scoped and paginated.

6.1 Dashboards / Analytics (/dashboard/analytics)

Purpose: Program & portfolio analytics for the active branch.

UI: A greeting banner, then a tabbed dashboard with seven views: *Program · Businesses/Projects · Investment · Learning · Team · Program Reports · Entrepreneur Learning Insights*. A row of global filters sits at the top: **Branch/ Org, Program, Application Year**.

Program view (fully documented): a *Businesses/Projects* panel with filters (Stage of business/project, Industry, Applicant age, Applicant gender, Country, Province, District) and:

- A funnel bar chart: **Applied / Selected / Graduated** counts (e.g. 80 / 79 / 0).
- **Graduation rate (%)**, **Selected businesses**, **Graduated businesses** tiles.
- **Graduation time:** min / average / max days.
- Distribution breakdowns with counts: **Stage of Business** (Ideation, SGB, Startup), **Industry** (17 industries), **Gender**, **Age band** (<17, 18–24, 25–34, 35–44, 45–54, 55–64, 65+), **Country / Province / District**.

The other six tabs are analogous aggregations (investment metrics, learning/course completion, team, program-level reports, and entrepreneur learning insights). Each is a read-only analytics view driven by its own dashboard endpoint.

API: GET /dashboard-program/ , GET /dashboard-businesses/ (branch-scoped); plus reference lists for the filters (genders , business-types , business-industries , countries , program-list , applicants/{branch}/ application-years).

Replica requirement: implement each dashboard view as an aggregation endpoint returning the counts/tiles/ breakdowns above; filters are query parameters. Charts: horizontal bar (funnel), donut (graduation rate), stat tiles, and category-count lists.

6.2 Announcements (/announcements)

Purpose: Branch-wide announcements/notices to entrepreneurs & staff.

UI: table of *Announcement title · Date · Reactions (count) · Status · Action*; search; filter by status; **Create announcement**. Announcements can be **Published/Draft**, can target programmes, and support **reactions** (a lightweight social acknowledgement, count shown).

API: GET/POST /announcements/{branch}/ , GET /announcement-statuses/ , GET /programs/{branch}/ .

6.3 Selection Board / Applicants (/selections/investment-board)

Purpose: The applicant selection pipeline — the heart of intake.

UI: stage sub-tabs (**All applicants** + one per stage: *Track I Onboarding*, *Track I Support*, *Pitch Stage*, *Track II Onboarding*, *Track II Support*) plus **Manage stages**. A filter bar: Program, Average score, Status, Country, Province, District, Registered. Table columns: *Business/Project · (Founder) · Program · Stage · Evaluated by me (Yes/No) · Status · Registered (Yes/No) · Average Evaluator Score · Application date · Actions*.

Features: - Move applicants between stages (drag or action). - Open an applicant to view their **submitted application answers** and to **score** them against the stage's Scoring Form (evaluator scoring; "Evaluated by me" tracks whether the current user has scored). - Set **status** (Invited / In Selection / Rejected / Archived). - Invite/register founders (sends them a login; **registered** flips true once they sign up). - Configurable stages via **Manage stages** (ordered, named, described).

API: GET /v2/applicants/{branch}/ (paginated, **search**), GET /stages/{branch}/ , GET /applicant-statuses/ , GET /users/{branch}/mentors/ , GET /countries/selected_countries|selected_provinces|selected_districts/{branch}/ .

6.4 Forms (/forms)

Purpose: Build the **Application Forms** (what applicants fill in) and **Scoring Forms** (what evaluators score with). Two sub-tabs: *List of application forms* and *List of scoring forms*.

Application form builder. A form has: name (multi-language), main language, description, a linked **Program**, a **status** (Draft/Published), and an ordered list of **fields**. Each field has a **field type**, **is_required**, **order**, multi-language label, and (for choice types) **options**. Observed field types:

Field type	code_name	Behaviour
Header	header	Section heading / non-input divider
Input	input	Free text input
Poll	poll	Single choice (radio) with options
Multiply Poll	multi_poll	Multiple choice (checkboxes) with options
Spinner	spinner	Dropdown/select with options

The replica should also support **file upload**, **date**, **number**, and **long text** field types (needed for real onboarding forms — e.g. document uploads for "Identity Card or Passport"). Treat the five observed types as the minimum and add these.

Also on the Forms screen: **Manage channels** (application source channels) and **Manage subindustries** (taxonomy management).

Scoring form builder. A scoring form is tied to a **Program** and a **selection stage**, has an **is_for_graduation** flag (so the same builder produces both selection-scoring and graduation-scoring forms), and holds **scoring questions**, each with: name, description, **weightage (%)**, **is_required**, **order**. The evaluator's answers combine (weighted) into the applicant's *Average Evaluator Score*.

API: GET/POST /application-forms/{branch}/ , GET /application-forms/{branch}/{id}/ , GET/POST /scoring-forms/{branch}/ , GET /scoring-forms/{branch}/{id}/ , GET /form-statuses/ .

6.5 Programs (/programs)

Purpose: Define cohorts/programmes and manage everything inside them.

UI: **Program types** cards (e.g. *Track I Support – 6/8 Months*, *Track II Support – 12 Months*) with counts + **Manage types**; a programmes table: *Program · Program type · Businesses/Projects (count) · Courses (count) · Creation date · Scoring required (Yes/No) · Actions (edit/delete)*; **Add program**; search.

Program detail (/programs-courses/{id}/) — tabbed: - **Program Businesses/Projects** — table of enrolled businesses: *Business/Project/Team name · Founder(s) · Course progress % · Course score % · Average evaluator score % · Mentors · Graduation (Graduated/Not graduated toggle) · Actions (Invest)*. Search + **Export to Excel**. - **Business/Project files** — documents per business. - **Courses progress** — per-course completion across the cohort. - **Mentor's review questions** — the question set mentors answer when reviewing businesses. - **Mentor's Conclusion Questions** — the question set for mentor final conclusions.

Features: create/edit programmes and program types; enrol businesses; assign courses; toggle graduation; export cohort data; launch investment ("Invest") for a business.

API: GET/POST /programs/{branch}/ , GET /programs/{branch}/{id}/ , GET /program-statuses/{branch}/ , GET /v2/programs/{branch}/{program}/businesses/ (paginated, business_name), GET /programs/{branch}/{program}/courses/ , GET /v2/programs/{program}/business-review/questions/ , GET /v2/programs/{program}/mentor-conclusion/questions/ .

6.6 Courses / LMS (/courses)

Purpose: Full learning-management system delivering training to entrepreneurs.

UI: courses table: *Course · Status · Modules (count) · Language · Subtitle languages · Actions*; search; language filter; **Add course**. Courses may be branch-wide or attached to a Program.

Structure: Course → Modules → Lessons → Content blocks . - **Course:** name, description, main language, subtitle_languages[] , status (Draft/Published), modules_count , and per-user progress , is_started , is_completed , completed_modules_count . - **Module:** name, description, ordered; per-user progress , is_completed . - **Lesson:** name, description, ordered; per-user is_completed ; a list of **content blocks** . - **Content block:** typed content. Observed type: **video**. The replica should support at least: *video, rich text/HTML, file/download, image, and quiz/assessment* blocks.

Multi-language: courses carry a language plus subtitle languages (observed: English, Русский/Russian, Arabic). Content and UI are localised (see Section 10).

Progress tracking: per entrepreneur, per content/lesson/module/course, aggregated up. This feeds the program's "Course progress %" and the Learning dashboards.

API: GET/POST /courses/{branch}/ , GET /courses/{branch}/{id}/ (returns nested modules → lessons → content with per-user progress), GET /course-statuses/ .

6.7 Library (/library)

Purpose: Document repository. Two sub-tabs: **Branch files** and **By program**. **Upload File**. Files are stored per branch and optionally per program (e.g. "Finnpact Business Planning Guide").

API: GET/POST /branch/{branch}/documents/ .

6.8 Investment Forms (/investment-forms)

Purpose: Forms specific to the investment track (e.g. "OCIF Track II Application Form"). Same builder model as application forms (name, description, status Draft/Published, ordered typed fields) but scoped to the investment process rather than programme intake.

API: GET/POST /investment-application-forms/{branch}/ , GET /investment-application-forms/{branch}/{id}/ , GET /investment-form-statuses/ .

6.9 Dealflow (/deal-flow)

Purpose: Investment pipeline (CRM of investment cases).

UI: stage sub-tabs (**All deals** + *Application, Evaluation, Pitching, Selection, Approved*) + **Manage stages**; filters: Tier, Stage, Type, Round, Industry; **Add investment case**; table of investment cases.

Investment case carries at minimum: business/company, type (business type), industry, **tier** (1–3), **round** (Pre-seed, Seed, Seed plus, Series A/B/C, IPO), **stage**, **status**, financials, and links back to the originating business/program.

API: GET/POST `/investment/{branch}/dealflow/investment-cases/`, GET `/investment-stages/{branch}/`, GET `/investment-statuses/`, GET `/investment-tiers/`, GET `/investment-rounds/`, GET `/business-industries/`, GET `/business-types/`.

6.10 Approval (`/approval`)

Purpose: Route investment cases through **approval committees** (multi-level).

UI: **All deals** + committee sub-tabs (e.g. *OCIF Track II Selection Committee – Cohort 3 – 2026*) + **Manage committees**; filters: Type, Round, Tier, Status, Industry; table of cases awaiting decisions.

Model: **Committee levels** are configurable per branch; a case moves through levels and gathers decisions; investment **status** cycles through *In Approval* → *Revision* / *Approved* / *Rejected*.

API: GET `/committee-levels/{branch}/`, GET `/investment/{branch}/approval/investment-cases/` (paginated), plus the investment reference lists.

6.11 Portfolio management (`/portfolio-management`)

Purpose: Manage approved investments as a portfolio. Table of portfolio companies/cases with filters (Type). Feeds the financial reports.

API: GET `/investment/{branch}/portfolio_management/investment-cases/` (paginated), GET `/business-types/`.

6.12 Reports (`/reports`)

Purpose: Investment/financial reporting pack, **Export to Excel** throughout, with a date selector.

Report types (sub-tabs): *Detailed information* · *Portfolio snapshot* · *Payments schedule* · *Aging analysis* · *Forex*.

Detailed information is itself sectioned into: *Overview* · *Finance timeline* · *Investment metrics* · *Collateral* · *CEO profile* · *Business/Project details* · *Co-financing summary* · *Financial Indicators* · *Sustainability & Innovation* · *Equity overview* · *Requested technical assistance*. "Finance timeline" supports start/end date ranges and totals (Total investment amount, and in USD → implies FX conversion).

API: GET `/branch/{branch}/reports/detailed-info/{section}/` for each section (`overview`, `financial-timeline`, `investment-metrics`, `collateral-overview`, `ceo-profile`, `business-details`, `co-financing-summary`, `financial-indicators`, `sustainability`, `equity-overview`, `requested-technical-assistance`), each also supporting `?page=1&page_size=N` for export. Plus endpoints for portfolio-snapshot, payments-schedule, aging-analysis and forex.

Replica requirement: every report needs a paginated JSON endpoint plus an Excel export (server-side `openpyxl` / `pandas` — you already run `pandas` in `FinnPayments`).

6.13 Calendar (`/apps/calendar`)

Purpose: Shared meetings/events. **Add meeting**; **Month/Week/Day/List** views; filter **All** / **Public** / **Private**. Events have visibility (public/private), presumably attendees, time, description.

API: GET/POST `/calendar-events/{branch}/?type=`.

6.14 Directories (`/directories`)

Purpose: People directory across the branch (entrepreneurs, mentors, staff). Table: *Name* · *Company* · *Position* · *Role*; search; filter by role. 107 people in the studied branch.

API: GET /v2/users/{branch}/?ordering=first_name (paginated), GET /roles/{branch}/.

6.15 Chat (/apps/chat)

Purpose: In-app 1:1 / group messaging between users (staff ↔ entrepreneurs ↔ mentors), scoped to programmes. The bell in the top bar shows unread notifications.

API: POST /user-chats/ (create/list chats), GET /users/{branch}/, GET /programs/{branch}/. **Replica**

requirement: real-time delivery via **WebSocket** (recommended) or polling; message persistence; unread counts; per-user threads.

6.16 Users & Roles/Permissions (/administration/roles-permissions/)

Purpose: User administration and RBAC. Table: *Name · Position · Role · Status (Active/Invited) · Actions*; search; filter by status & role; **Add user** (invite by email); **Manage roles**.

Roles are branch-scoped, each with `name`, `code_name`, `is_constant`. Constant roles (Mentor, Org Admin, Entrepreneur, Administrator) always exist; custom roles (e.g. Investment Manager) can be created and are assigned granular permissions. See Section 9.

API: GET /v2/users/{branch}/ (paginated), GET/POST /users/..., GET /roles/{branch}/, GET /user-roles/{branch}/, GET /user-statuses/.

6.17 Organizations (/administration/organizations)

Purpose: Manage organizations (and, by extension, their branches). Table: *Name · Date of registration · Status · Actions*; search; status filter. Top of the tenancy tree.

API: GET/POST /organizations/, GET /organization-statuses/, GET /user-branches/.

6.18 Help Center (/help-center/)

Purpose: In-app help/documentation. Static/CMS-like help content for users.

7. Data model

Below is the entity model inferred from the live API. IDs on the original are large integers (snowflake- style, e.g. 892348233054); on the replica use standard auto-increment or UUID/BIGINT — see Section 12. `OB` = `organization_branch` (the tenant key on almost every table).

7.1 Tenancy & identity

- **Organization** (`id`, `name`, `date_of_registration`, `status`)
- **OrganizationBranch / Branch** (`id`, `organization_id`, `name`, ...) — the tenant scope for all data below.
- **User** (`id`, `email`, `first_name`, `last_name`, `photo`, `position`, `status`[Active|Invited], ...)
- **Role** (`id`, `OB`, `name`, `code_name`, `is_constant`)
- **UserRole / membership** — links User ↔ Role ↔ Branch (a user can belong to multiple branches/roles).
- **Permission & RolePermission** — granular capability grants per role (see Section 9).

7.2 Intake & selection

- **ApplicationForm** (`id`, `OB`, `program_id`, `name`{+ per-language}, `main_language`, `form_description`, `status`)
- **ApplicationFormField** (`id`, `application_form_id`, `name`{+lang}, `field_type_id`, `is_required`, `order`)
 - **ApplicationFormFieldOption** (`id`, `application_form_field_id`, `name`{+lang})
- **FieldType** (`id`, `name`, `code_name`) — header/input/poll/multi_poll/spinner/(file/date/number/...)

- **Stage / SelectionStage** (id, 0B, name, order, description)
- **Applicant** (id, 0B, program_id, selection_stage_id, status, email, first_name, last_name, business_name, business_logo, country_id, average_score, registered, evaluated_by_me, created_date, ...)
- **ApplicationSubmission / Answers** — the applicant's answers to an ApplicationForm's fields.
- **ScoringForm** (id, 0B, program_id, selection_stage_id, is_for_graduation, name, status)
- **ScoringFormField / ScoringQuestion** (id, scoring_form_id, name, description, weightage, is_required, order, field_type)
- **Evaluation / Score** (id, scoring_form_id, applicant_id, evaluator_user_id, answers, total_score)
→ aggregated to Applicant.average_score.
- **ApplicantStatus** (Invited, In Selection, Rejected, Archived)
- **Channel** (application source) and **Subindustry** (taxonomy) — managed from Forms.

7.3 Programmes & delivery

- **ProgramType** (id, 0B, name, duration) — e.g. "Track I Support – 6 Months".
- **Program** (id, 0B, program_type_id, name, status, scoring_required, creation_date)
- **Business / Project** (id, 0B, program_id, name, founders[], mentors[], stage, graduation_status [Graduated|Not graduated], course_progress, course_score, average_evaluator_score, ...) — a selected Applicant becomes a Business in a Program.
- **BusinessType** (Startup, Ideation Stage, SGB) ; **BusinessIndustry** (17 industries).
- **BusinessFile / Document** — files per business/branch/program (Library + program "Business files").
- **MentorReviewQuestion** (id, program_id, ...) and **MentorConclusionQuestion** (id, program_id, ...) with mentor **answers** per business.
- **Mentorship** — links Mentor(User) ↔ Business.

7.4 Learning (LMS)

- **Course** (id, 0B, program_id?, name, description, language, subtitle_languages[], status)
- **Module** (id, course_id, name, description, order)
 - **Lesson** (id, module_id, course_id, name, description, order)
 - **ContentBlock** (id, lesson_id, type[video|text|file|image|quiz], payload/url, order)
- **CourseEnrollment / Progress** (user_id, course_id, is_started, is_completed, progress, ...) and the same at module/lesson/content granularity.

7.5 Investment

- **InvestmentForm** (id, 0B, name, form_description, status, fields[]) (same field model as app forms).
- **InvestmentCase** (id, 0B, business_id/company, type, industry, tier[1-3], round[Pre-seed...IPO], stage[Application...Approved], status[In Approval|Revision|Approved|Rejected], financials{...}, ...)
- **InvestmentStage**, **InvestmentStatus**, **InvestmentTier**, **InvestmentRound** — reference/enums (stages configurable per branch via "Manage stages").
- **Committee & CommitteeLevel** (id, 0B, name, order) and **CommitteeDecision** (committee_level_id, investment_case_id, decision, notes, decided_by).
- **PortfolioEntry** — an approved InvestmentCase under active management, plus its financial records: finance timeline, investment metrics, collateral, co-financing, financial indicators, equity, payments schedule, aging, forex, technical-assistance requests (each a related table feeding Reports).

7.6 Collaboration & misc

- **Announcement** (id, 0B, title, body, program_id?, status, date, reactions_count) + **AnnouncementReaction** (announcement_id, user_id, type).
- **CalendarEvent** (id, 0B, title, start, end, visibility[public|private], attendees[], description).
- **Chat** (id, participants[], program_id?) + **ChatMessage** (chat_id, sender_id, body, sent_at, read_by[]).
- **Notification** (user_id, type, payload, read) — drives the top-bar bell.
- **Country / Province / District** — geographic reference (with "selected_" filters per branch), Gender* reference.

7.7 Reference/enumeration tables (seedable)

FieldType, FormStatus, CourseStatus, ProgramStatus, ApplicantStatus, UserStatus, OrganizationStatus, AnnouncementStatus, InvestmentStatus, InvestmentTier, InvestmentRound, InvestmentStage, BusinessType, BusinessIndustry, Gender, Country/Province/District. (Values catalogued in Appendix 15.3.)

8. API design & conventions

The replica's API should mirror the original's shape so the React client can be ported with minimal change.

Observed conventions:

- **Base path & i18n prefix:** `/lang/api/...` (e.g. `/en/api/programs/...`). The language segment selects response localisation. Keep this: route `/lang/api/...` and use `lang` for i18n of enum/labels.
- **Versioning:** a mix of unversioned (`/api/...`) and `/api/v2/...` for newer/paginated list endpoints (applicants, users, program businesses). Adopt `/api/v2/` for list endpoints with pagination/filtering.
- **Tenancy in the path:** branch id is a path segment on branch-scoped resources (`/api/programs/{branch_id}/...`) — the server must still authorise that the caller may access that branch.
- **Pagination:** `?page=&page_size=` (DRF-style). Responses for v2 lists are `{count, next, previous, results[]}` (implement the same envelope). `page_size=1000000` is used to fetch "all" for exports — provide a proper export endpoint instead in the replica to avoid unbounded queries.
- **Filtering/search:** query params (`search`, `business_name`, `type`, `ordering`, plus domain filters).
- **Errors:** DRF-style — `{"detail": "..."}` with appropriate status (e.g. 401/403 "You do not have permission to perform this action.").
- **Statuses/enums as objects:** returned as `{id, name, code_name}` (client filters on `code_name`).
- **Nested reads:** detail endpoints return nested children (course → modules → lessons → content; form → fields → options) — convenient for the SPA; replicate.

The full observed endpoint catalogue is in **Appendix 15.1**.

9. Authentication & authorization

9.1 Mechanism (observed)

- **JWT (HS256) access token + refresh token**, stored in the browser's `localStorage` (`accessToken`, `refreshToken`), plus `currentBranchId` and a `userData` blob.
- Requests authenticate with **Authorization: Bearer <accessToken>**.
- Login exchanges credentials for the token pair; a refresh endpoint issues new access tokens.

Security defect in the original — do NOT replicate. The original's JWT **payload contains the user's password** (alongside `user_id` and an expiry). That is a serious flaw: JWT payloads are only base64-encoded (not encrypted), so anyone who reads the token reads the password. In the replica: **never put the password (or any secret) in the JWT**. Put only `user_id`, `branch_id`, roles, token type, and expiry. Store password hashes only (argon2/bcrypt). Prefer **httpOnly, Secure, SameSite cookies** for the refresh token over `localStorage` to reduce XSS exposure; keep the short-lived access token in memory. This is the single most important correction to make versus the original.

9.2 Authorization (RBAC)

- Roles are **branch-scoped** with `code_name` (`branch_admin`, `organization_admin`, `entrepreneur`, `mentor`, `investment_manager`, + custom). Some are `is_constant` (cannot be deleted).
- Permissions are **granular** and assigned per role via **Manage roles** — every module/action should map to a permission (e.g. `programs.view`, `programs.edit`, `applicants.score`, `investment.approve`, `users.manage`, `reports.view`, ...). The API must enforce them server-side, not just hide UI.
- Every request is authorised on two axes: **(a) does the user's role grant the capability**, and **(b) may the user act within the target `branch_id`** (tenant isolation).
- The default landing/experience differs by role (staff dashboard vs entrepreneur portal).

10. Internationalization (i18n)

- **UI language switcher** in the top bar; API takes a `/{{lang}}/` path segment.
- **Content is multilingual at the field level:** form/field/option names carry per-language columns — observed suffixes: `_en`, `_ru`, `_uz`, `_ky`, `_tg`, `_ur`, `_ar`, `_fa`, `_pt` (English, Russian, Uzbek, Kyrgyz, Tajik, Urdu, Persian/Farsi, Portuguese) plus a `main_language`. Courses carry a language and `subtitle_languages[]`.
- **RTL support required** (Arabic, Urdu, Farsi are present in real data).
- **Replica requirement:** store translatable strings as per-language columns or a translations table; the React app uses an i18n library (e.g. `react-i18next`); the FastAPI layer returns the requested language's variant and falls back to `main_language`.

11. Non-functional requirements

- **Multi-tenancy & isolation:** strict per-branch scoping and authorization on every query.
- **File storage:** documents, business files, course videos, logos/photos. Videos are large — store on disk (or object storage) and stream; do not put binaries in the DB. Serve via nginx `X-Accel-Redirect` or signed URLs.
- **Exports:** Excel export on programmes and all reports.
- **Pagination everywhere** for lists (some branches have 100+ users, 80+ applicants).
- **Real-time** for chat and notifications (WebSocket recommended).
- **Observability:** the original uses **Sentry** (React SDK) for error tracking — keep an equivalent (self-hosted GlitchTip/Sentry or logs).
- **Audit trail** for selection decisions, scoring, graduation, and investment approvals (governance).
- **Performance:** dashboard aggregations should be indexed/materialised; avoid `page_size=1000000` scans.
- **Backups:** since data includes grant/investment records, schedule DB + file backups.
- **Security:** HTTPS only (Certbot), input validation (pydantic), rate-limiting on auth, RBAC enforced server-side, secrets in env not code, and the JWT correction from §9.1.

12. Recommended technology stack (matches FinnPayments)

Chosen to slot in beside FinnPayments/FinnVerify on the same VPS with the same operational model.

Backend - Python 3 + FastAPI + uvicorn (single `run.py` entrypoint, same as FinnPayments), **pydantic v2** for schemas/validation. - **SQLAlchemy 2.x** ORM + **Alembic** migrations. **pandas/openpyxl** for the Excel exports and any analytics crunching (you already ship pandas/scikit-learn in FinnPayments). - **Auth:** `python-jose / pyjwt` for JWT, `passlib[argon2]` for password hashing, `fastapi` dependencies for RBAC + tenant guards. Refresh token in httpOnly cookie (see §9.1). - **Real-time:** FastAPI WebSocket endpoints for chat/notifications (no extra infra needed initially). - **Background jobs (optional):** start with FastAPI `BackgroundTasks` ; add a worker later if needed. Note: Redis already exists on the box (used by other apps) if you later want a queue — keep it namespaced.

Database - SQLite to match FinnPayments' default (`DATABASE_URL` → a file in the project dir, e.g. `virtualaccelerate.db`). This is fine to start. - **Recommendation:** this app is more relational and concurrent (chat, many writers, aggregations, investment financials) than a single-user tool. **Design against `DATABASE_URL` so you can move to PostgreSQL 16** (already installed on the box for n8n) with no code change. If you expect concurrent writers or heavy reporting, start on Postgres. Either way, keep everything behind SQLAlchemy so the swap is a connection-string change.

Frontend - React 18 + Vite 5 (same as FinnPayments; the original is also React, so components/UX port closely). - Router with SPA history-fallback; an i18n library with RTL; a component/chart library for the dashboards (e.g. MUI + Recharts/ECharts). State/data layer: React Query (TanStack) against the REST API. - Auth: access token in memory, refresh via cookie; **Authorization: Bearer** on API calls; `currentBranchId` kept in app state (not trusted by the server — server re-authorises).

Edge - nginx reverse proxy + **Certbot/Let's Encrypt** TLS, subdomain-per-app model (see §13).

This mirrors FinnPayments exactly: systemd-managed FastAPI on a private port, Vite/React on another private port (dev) or a built static bundle (recommended for prod — see §13.3), nginx terminating TLS and proxying.

13. Deployment on the VPS (Ubuntu 24.04, alongside FinnPayments/FinnVerify)

13.1 Topology

Add a new subdomain for the app, following the sibling pattern (`payments.finnverify.com` , `screen.finnverify.com` , ...). Suggested: **`accelerate.finnverify.com`** (or a domain of your choice). Everything runs as **systemd services under the administrator user**, no containers, consistent with FinnPayments.

```
accelerate.finnverify.com —nginx(:443, Certbot)→ 127.0.0.1:3002 (frontend: static build or Vite)
                                     ↳ /api,/auth,/ws,/media...
                                     proxied ▶ 127.0.0.1:8002 (FastAPI/uvicorn)
```

Pick free local ports so as not to collide with existing apps (FinnPayments uses 3001/8001; screen/aml use 8000; risk 8080; n8n 5678; esign 8088/3003). Suggested here: **backend `127.0.0.1:8002` , frontend `127.0.0.1:3002` .**

13.2 Backend service (systemd) — mirrors `finnpayments-backend.service`

`/etc/systemd/system/virtualaccelerate-backend.service :`

```

[Unit]
Description=VirtualAccelerate Backend (FastAPI/uvicorn)
After=network.target

[Service]
User=administrator
WorkingDirectory=/home/administrator/virtualaccelerate
ExecStart=/usr/bin/python3 run.py
Environment=PORT=8002
Environment=DATABASE_URL=sqlite:///home/administrator/virtualaccelerate/virtualaccelerate.db
Environment=JWT_SECRET=<set-a-strong-secret>
Restart=on-failure

[Install]
WantedBy=multi-user.target

```

`run.py` boots uvicorn on `127.0.0.1:8002` (bind localhost only; nginx is the edge). Keep `DATABASE_URL`-driven config so Postgres is a drop-in later.

13.3 Frontend service

Two options — recommend **(A) for production**:

(A) Built static bundle (recommended). `npm run build` → serve the `dist/` via nginx directly (fastest, no Node process). No frontend systemd service needed; nginx serves static files with SPA fallback.

(B) Vite dev server as a service (mirrors FinnPayments' current setup). `/etc/systemd/system/virtualaccelerate-frontend.service` running `npm run dev -- --port 3002 --host 127.0.0.1`. Matches how FinnPayments runs today, but a dev server in prod is not ideal — use (A) unless you specifically want parity with the current FinnPayments approach.

13.4 nginx site

`/etc/nginx/sites-available/accelerate.finnverify.com` (enable via symlink, then `certbot --nginx`):


```

server {
    server_name accelerate.finnverify.com;

    # --- API, auth, websockets, media → FastAPI ---
    location ~ ^/(api|auth|ws|media|static-api)/ {
        proxy_pass http://127.0.0.1:8002;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        # websocket upgrade for /ws (chat/notifications)
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
    }

    # --- Frontend ---
    # Option A (static build):
    root /home/administrator/virtualaccelerate-frontend/dist;
    index index.html;
    location / { try_files $uri $uri/ /index.html; } # SPA history-fallback

    # Option B (Vite dev): replace the two lines above with:
    # location / { proxy_pass http://127.0.0.1:3002; proxy_set_header Host $host; }

    listen 443 ssl; # managed by Certbot
    # ssl_certificate ... (Certbot fills these in)
}
server {
    listen 80;
    server_name accelerate.finnverify.com;
    return 301 https://$host$request_uri; # Certbot adds this
}

```

The original strips the `/api` prefix at the proxy (Vite does it there). In the replica, either keep the `/api` prefix all the way to FastAPI (simplest — mount routers under `/api`), or strip it in nginx with a rewrite. Pick one and be consistent so the React client's base URL is correct.

13.5 DNS & TLS

- Add an A record for `accelerate.finnverify.com` → the VPS IP.
- `sudo certbot --nginx -d accelerate.finnverify.com` to issue/renew TLS (same as siblings).

13.6 Media/large files

Course **videos** and documents can be large. Serve them through FastAPI-issued signed URLs or nginx `internal` locations with `X-Accel-Redirect`, storing files under `/home/administrator/virtualaccelerate/media/`. Keep them out of the DB and out of the git repo.

13.7 Operational parity checklist

- Runs as `administrator` via systemd, `Restart=on-failure` ✓
- Binds to localhost, nginx terminates TLS ✓
- `DATABASE_URL` -driven (SQLite now, Postgres-ready) ✓
- Certbot-managed certs ✓
- Subdomain-per-app ✓
- Logs to journald (`journalctl -u virtualaccelerate-backend`) ✓

14. Suggested build roadmap (phased)

Build in the order the funnel flows so each phase is demoable.

Phase 0 — Foundations. Repo scaffold (FastAPI `run.py`, React/Vite), SQLAlchemy + Alembic, JWT auth (corrected), RBAC + tenant guards, Organizations/Branches/Users/Roles, i18n plumbing, nginx+systemd deploy. *Exit: a user can log in, switch branch, and see an empty shell.*

Phase 1 — Intake & selection. Application form builder + public form rendering + submission → Applicant; Selection Board (stages, filters, statuses); Scoring forms + evaluator scoring → average score. *Exit: run a real intake and select a cohort.*

Phase 2 — Programmes & LMS. Program types & programmes; enrol businesses; Courses (modules/lessons/ content, video, progress); program detail tabs; mentor review & conclusion questions; graduation. Library. Announcements + reactions. *Exit: deliver a cohort end-to-end.*

Phase 3 — Investment. Investment forms; Dealflow (stages); Approval (committees, multi-level); Portfolio management; the Reports pack + Excel export. *Exit: promote a graduate to an investment and manage it.*

Phase 4 — Collaboration & analytics. Dashboards (all 7 views), Calendar, Chat + notifications (WebSocket), Directories, Help Center, Sentry-equivalent monitoring. *Exit: feature parity.*

Phase 5 — Hardening. Audit trails, backups, performance/indexing, RTL polish, security review, and (if needed) Postgres migration.

15. Appendices

15.1 Observed API endpoint catalogue

All under `https://app.virtualaccelerate.com/{lang}/api/`. `{b}` = branch id. Methods shown are those inferred/observed; assume standard REST (GET list/detail, POST create, PUT/PATCH update, DELETE) unless noted.

Identity / tenancy - `GET user-branches/` — branches the current user can access - `GET users/{b}/{userId}/` — current user context - `GET v2/users/{b}/?ordering=&page=&page_size=` — users list (Directory/Users) - `GET users/{b}/mentors/` — mentors in branch - `GET roles/{b}/` · `GET user-roles/{b}/` · `GET user-statuses/` - `GET organizations/` · `GET organization-statuses/`

Reference / enums - `GET genders/` · `GET countries/` · `GET countries/selected_countries|selected_provinces|selected_districts/{b}/` - `GET business-types/` · `GET business-industries/` - `GET form-statuses/` · `GET course-statuses/` · `GET program-statuses/{b}/` · `GET announcement-statuses/` - `GET applicant-statuses/` · `GET investment-statuses/` · `GET investment-tiers/` · `GET investment-rounds/` - `GET investment-stages/{b}/` · `GET stages/{b}/` · `GET committee-levels/{b}/` - `GET investment-form-statuses/`

Dashboards - `GET dashboard-program/?branch_id={b}` · `GET dashboard-businesses/?branch_id={b}` - `GET program-list/?branch_id={b}` · `GET v2/applicants/{b}/application-years/?branch_id={b}`

Forms - `GET/POST application-forms/{b}/` · `GET application-forms/{b}/{id}/` - `GET/POST scoring-forms/{b}/` · `GET scoring-forms/{b}/{id}/` - `GET/POST investment-application-forms/{b}/?page=&page_size=` · `GET investment-application-forms/{b}/{id}/`

Selection - `GET v2/applicants/{b}/?page=&page_size=&search=` · applicant detail + scoring endpoints

Programmes - `GET/POST programs/{b}/` · `GET programs/{b}/{id}/` · `GET program-statuses/{b}/` - `GET v2/programs/{b}/{program}/businesses/?page=&page_size=&business_name=` - `GET programs/{b}/`

`{program}/courses/` - GET `v2/programs/{program}/business-review/questions/` - GET `v2/programs/{program}/mentor-conclusion/questions/`

Courses / LMS - GET/POST `courses/{b}/` · GET `courses/{b}/{id}/` (nested modules → lessons → content + progress) - GET `course-statuses/`

Library - GET/POST `branch/{b}/documents/`

Investment - GET/POST `investment/{b}/dealflow/investment-cases/` - GET `investment/{b}/approval/investment-cases/?page=&page_size=` - GET `investment/{b}/portfolio_management/investment-cases/?page=&page_size=`

Reports (each also `?page=&page_size=` for export) - GET `branch/{b}/reports/detailed-info/{section}/` where `{section}` ∈ `overview`, `financial-timeline`, `investment-metrics`, `collateral-overview`, `ceo-profile`, `business-details`, `co-financing-summary`, `financial-indicators`, `sustainability`, `equity-overview`, `requested-technical-assistance` - (+ `portfolio-snapshot`, `payments-schedule`, `aging-analysis`, `forex report endpoints`)

Collaboration - GET/POST `announcements/{b}/` - GET/POST `calendar-events/{b}/?type=` - POST `user-chats/` (+ message endpoints)

15.2 Field types (form builder)

`header` (Header, non-input) · `input` (text) · `poll` (single-choice radio, has options) · `multi_poll` ("Multiply Poll", multi-choice checkboxes, has options) · `spinner` (dropdown, has options). Scoring forms use a `scoring_form` field type ("Scoring Question") with a `weightage` (%). **Add for the replica:** `file` (uploads), `date`, `number`, `textarea/long_text`.

15.3 Enumeration values (as seen in the Finnpact branch)

- **Selection stages:** Track I Onboarding · Track I Support · Pitch Stage · Track II Onboarding · Track II Support (*configurable per branch*)
- **Applicant statuses:** Invited · In Selection · Rejected · Archived
- **Program types:** Track I Support – 6 Months · Track II Support – 12 Months (*also "8 Months" seen*)
- **Business types:** Startup · Ideation Stage · SGB
- **Business industries (17):** Accommodation, Food & Beverage Services · Agroforestry · Crops & Horticulture · ECD & Education · Electric, Auto, Equipment, Chemicals Products · Entrepreneurship Skills · Environmental Services & Management · Fashion, Apparel & Crafts · Financial, Professional & Management Services · Healthcare & Allied Services · ICT & Digital Services · Livestock, Aquaculture & Apiculture · Other Professional Services · Processed Food · Public Administration, Community & Social Services · Soft & Employability Skills · Transportation & Travel · Wholesale & Retail
- **Investment stages:** Application · Evaluation · Pitching · Selection · Approved (*configurable*)
- **Investment statuses:** In Approval · Revision · Approved · Rejected
- **Investment tiers:** Tier 1 · Tier 2 · Tier 3
- **Investment rounds:** Pre-seed · Seed · Seed plus · Series A · Series B · Series C · IPO
- **Roles:** Administrator (`branch_admin`) · Organization Administrator (`organization_admin`) · Entrepreneur (`entrepreneur`) · Mentor (`mentor`) · Investment Manager (`investment_manager` , custom)
- **Course/content:** content block type `video` observed; languages incl. English, Arabic, Russian; subtitle languages supported.
- **User statuses:** Active · Invited

15.4 Tech fingerprint of the original (for reference only)

Backend behaves like **Django + Django REST Framework** (`/lang/api/` i18n URL prefix, DRF pagination and permission-denied messages, `{id,name,code_name}` enum serialisation). Frontend is **React** with **Sentry** error

tracking. IDs are large snowflake-style integers. The replica intentionally re-implements this on **FastAPI + React/Vite + SQLite/Postgres** to match your FinnPayments environment; API *shape* is preserved so the client port is straightforward.

15.5 Security note (must-fix vs original)

The original stores the **user password inside the JWT payload** and keeps tokens in `localStorage`. The replica must **never** embed passwords/secrets in JWTs, must store only password **hashes** (argon2/bcrypt), and should hold the refresh token in an **httpOnly Secure cookie**. See §9.1.

End of specification.